

Introducción a la Ingeniería de Requerimientos

Clase 1 — El Software: Concepto, Evolución y Cualidades

Material teórico completo — texto, imágenes y videos complementarios

Introducción

En nuestra primera clase, veremos el concepto de software, desde el punto de vista de un proceso y desde la visión como producto. Su evolución, sus características y otros conceptos que complementan la idea de software.

El software, si bien es un término y un concepto que podemos tener claro, vamos a verlo en la perspectiva de la ingeniería de requerimientos.

El software en el tiempo

La idea del software como programa de una computadora fue concebida por primera vez en el año 1935 por el matemático británico Alan Turing, hace casi 100 años.

Posteriormente, recién en el año 1958, el científico estadounidense John Tukey formuló la siguiente frase:

“It is better to solve the right problem approximately than solve the wrong problem exactly”

Para que no queden dudas: el software no son solo programas. El software es un conjunto de partes interrelacionadas que alcanzan algún objetivo.

Entender el concepto de software es un medio para poder construirlo, y ese es nuestro objetivo real. Nos interesa la imagen que nos hacemos del software y, sobre todo, cuál es su papel en un contexto mayor. Reducir el software a programas es una simplificación válida para una computadora, pero no para quienes lo van a construir.

Esta simplificación tiene una consecuencia negativa para el desarrollo de software en cuanto a la productividad de los desarrolladores y la calidad de los productos generados.

El software es:

- Alma y cerebro de una computadora
- Corporización de las funciones de un sistema
- El conocimiento capturado acerca de un área de aplicación
- Colección de los programas y datos necesarios para convertir a una computadora (de propósito general) en una máquina de propósito especial diseñada para una aplicación particular
- Información (documentación) producida durante el desarrollo de un sistema software-intensivo

El software es muchas cosas, pero todas son aspectos de la información. En definitiva, es una cuestión de semántica:

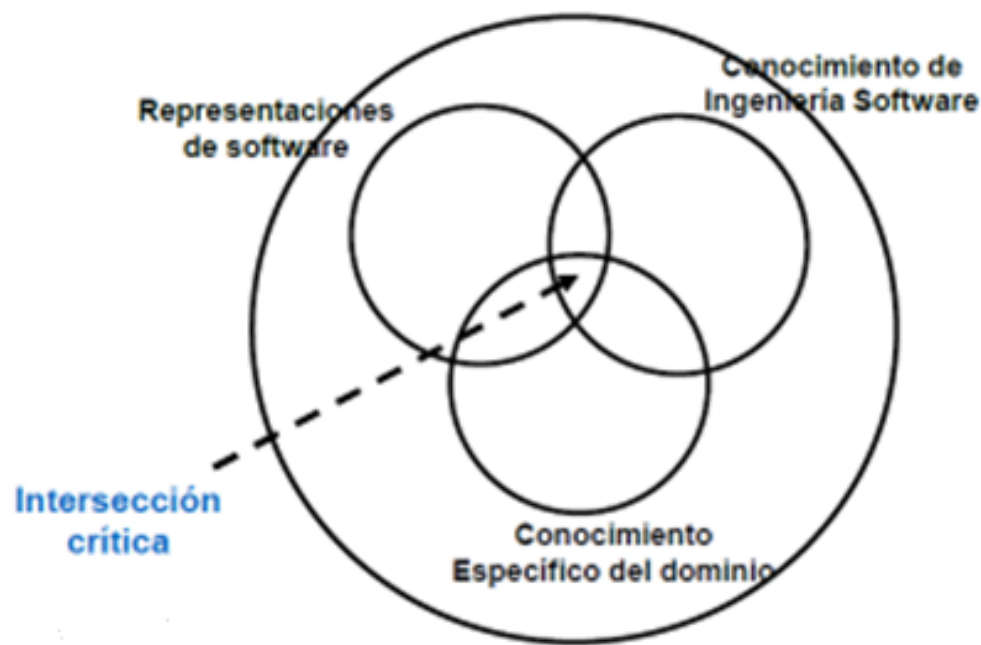
- Si software = programas ejecutables, excluimos una cantidad de información que debemos llamar de alguna manera.

- Si incluimos toda la información relevante a una pieza de software ejecutable, entonces debemos relacionarnos con esa información en la misma forma rigurosa y sistemática que lo hacemos con el software ejecutable.

Esto es crucial para un desarrollo exitoso, pues si no se hace, la información se pierde o se altera, introduciendo errores.

Video complementario sugerido para ampliar estos conceptos: ver sección "Videos complementarios" al final del documento (video 1).

Información en el ambiente de desarrollo



Intersección crítica entre representaciones de software, conocimiento de ingeniería de software y conocimiento específico del dominio.

Representaciones del software

Cualquier información que en forma directa representa un eventual conjunto de programas y los datos asociados. Incluye:

- Programas
- Diseños detallados
- Diseños de arquitectura (representados como diagramas de estructura)
- Especificaciones escritas en un lenguaje formal
- Requerimientos del sistema expresados en una combinación de notaciones
- O centenares de posibilidades

Conocimiento de ingeniería de software

Toda la información relativa al desarrollo en general (por ejemplo, cómo usar un método específico de diseño) o relativa a un desarrollo en particular (por ejemplo, un programa de testeo en un proyecto).

Incluye:

- Información relativa al proyecto
- Información sobre la tecnología de software (métodos, conceptos, técnicas)
- Conocimiento acerca de sistemas similares
- Información detallada relativa a la identificación y solución de problemas técnicos del sistema en desarrollo

Ejemplos de formas que toma el software

Colecciones de programas: a testear, terminados, en lenguaje de máquina, en lenguajes de alto nivel.

- Necesidades
- Requerimientos
- Especificaciones
- Diseños de arquitectura
- Diseños detallados
- Formatos de datos
- Sistemas en uso para producción
- Análisis de requerimientos
- Documentación del usuario
- Documentación de mantenimiento
- Pedidos de cambio
- Especificaciones de modificaciones
- Informes de errores
- Mediciones de performance

Punto de vista: estudiar el proceso de desarrollo tratando de ver sus formas en el viaje desde la idea hasta el producto.

Video complementario: ver sección "Videos complementarios" al final del documento (video 2).

¿El software es producto o conocimiento?

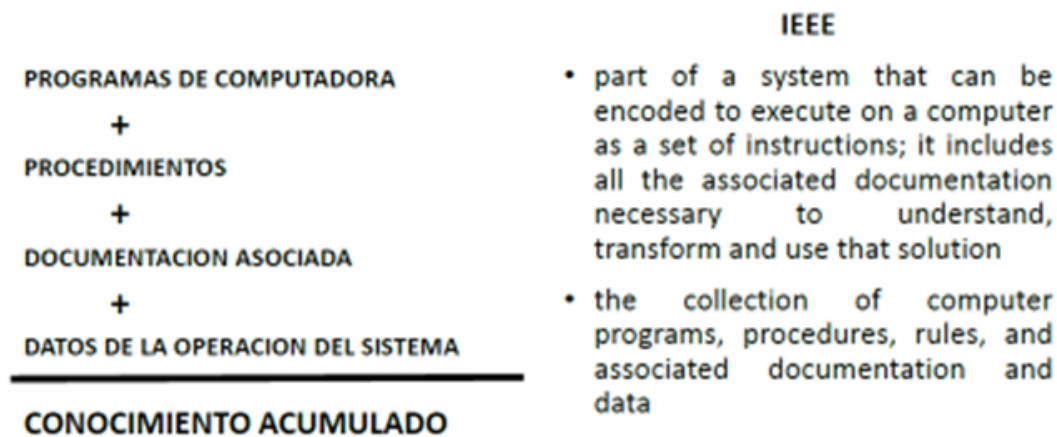
A medida que las computadoras extendieron su ámbito de aplicación se las vio como procesadores simbólicos, de allí que no se viera al software como producto. Desde la década de 1960 (con la separación entre software y hardware) comenzó a constituirse como producto. El software es tanto un producto como un objeto técnico, esto es: conocimiento empaquetado.

Razonamiento

- Los programas contienen conocimiento.
- El ejecutable es el final de una cadena de representaciones que llamamos software.
- Entonces, las versiones finales también contienen conocimiento.
- Perdemos ese conocimiento si nos limitamos a los conceptos del software ejecutable.

No perder este conocimiento es una de las principales motivaciones de la reusabilidad del software. El software se puede comparar con un informe de investigación.

Productos de software



Conocimiento acumulado = programas de computadora + procedimientos + documentación asociada + datos de la operación del sistema (definición IEEE).

¿Por qué el software es único?

- Es intangible: dificulta su control y medición.
- Posee un alto contenido intelectual.
- No se lo reconoce como un activo contable.
- Su proceso de desarrollo es mano de obra intensivo, basado en equipos y por proyectos. Esto multiplica el problema de la comunicación, un aspecto clave de los proyectos.
- No hay separación entre I+D y producción.
- Potencialmente es modificable hasta el infinito, lo que genera problemas a la hora de la evolución del software liberado.
- Otras cuestiones: degradación, costo de reproducción.

Características conceptuales

- El objetivo de la ingeniería es la construcción de un producto.

- El objetivo de la Ingeniería de Software es la construcción de sistemas de software.
- La principal diferencia es que el software es maleable.
- Existe la idea (errónea) de que los cambios en el software son fáciles.

En este punto debe pensarse en los productos de software de otra forma:

- Un cambio debe ser visto como un cambio en el diseño más que en el código.
- Su producción es humano-intensiva: requiere más ingeniería que manufactura. El proceso de producción de software se vincula más con el diseño e implementación que con la manufactura.

En la ingeniería clásica, el ingeniero dispone de herramientas para describir el producto que son distintas del producto en sí; no es así en la Ingeniería de Software.

Video complementario: ver sección "Videos complementarios" al final del documento (video 3).

Cualidades del software

A continuación se presentan las cualidades que debe cumplir el software:

- **Corrección funcional:** se comporta de acuerdo con la especificación de requerimientos funcionales.
- **Confiabilidad:** el usuario puede depender del software.
- **Robustez:** se comporta "razonablemente", incluso en circunstancias no previstas en la especificación de requerimientos.
- **Performance:** uso económico de los recursos de computación (en Ingeniería de Software se identifica con eficiencia).
- **Amistosidad:** fácil uso por parte de los seres humanos.
- **Verificabilidad:** sus propiedades pueden verificarse fácilmente.
- **Mantenibilidad:** puede repararse y evolucionar.
- **Reusabilidad:** permite utilizar sus componentes sin modificarlos en otros sistemas.
- **Portabilidad:** puede correr en distintos ambientes.
- **Comprensibilidad:** facilidad de ser entendido por los usuarios (desarrolladores).
- **Interoperatividad:** capacidad de coexistir y cooperar con otros sistemas.
- **Productividad:** mide la eficiencia del proceso de producción de software.
- **Oportunidad:** capacidad de liberar un producto a tiempo.
- **Visibilidad:** sus pasos previos y su estado actual están correctamente documentados.

Video complementario: ver sección "Videos complementarios" al final del documento (video 4).

Cualidades internas y externas

Las cualidades internas sólo son visibles para los desarrolladores. En cambio, las cualidades externas son visibles para los usuarios. Sin embargo, hay una fuerte relación entre unas y otras, aunque

debemos tener claro que las cualidades externas no son consecuencia directa de las internas.

Son las cualidades internas, vinculadas en gran medida con la estructura del software, las que ayudan a los desarrolladores a que se cumplan las cualidades externas. Por ejemplo, la cualidad interna de verificabilidad es necesaria para conseguir la cualidad externa de fiabilidad o confiabilidad. Sin embargo, estas cualidades están estrechamente relacionadas y la distinción entre internas y externas no es tan tajante.

Visiones según el rol

Es posible pensar en las cualidades requeridas en el software según tres perspectivas diferentes: el usuario final, el ingeniero de software (desarrollador/productor) y el project manager. Cada uno tiene intereses distintos:

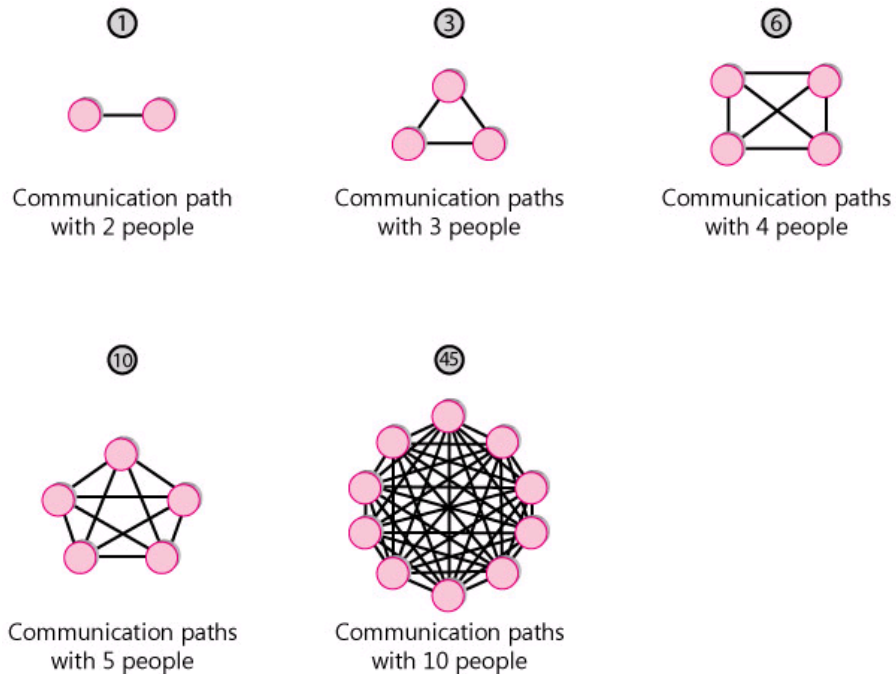
- **Usuario final:** pretende que el sistema posea corrección funcional, respete la especificación de requerimientos, sea confiable, robusto y fácil de usar.
- **Ingeniero de software / desarrollador:** le interesa que el software sea reusable, portable, comprensible internamente, que pueda interoperar con otros productos, que sea verificable y que se haya mantenido.
- **Project manager:** le interesan las características vinculadas con el proceso de producción de software, fundamentalmente la productividad, la visibilidad y la oportunidad.

El software y la deseconomía de escala

El costo que insume la producción del software, visto como un proceso económico, es una característica fundamental. En este tema normalmente hablamos de economías de escala: ¿cómo se posiciona el software dentro de una economía de escala?

El desarrollo de software es una actividad económica y, como tal, encaja en el análisis de una economía de escala. Las personas asumen naturalmente que un sistema 10 veces más grande que otro requerirá algo así como 10 veces más esfuerzo para construirlo.

La cuestión básica es que, en el ámbito del software, los proyectos de mayor envergadura requieren la coordinación entre grupos más grandes de personas, lo que exige mayor comunicación. A medida que aumenta el tamaño del proyecto, el número de vías de comunicación entre diferentes personas aumenta como una función cuadrática del número de personas en el proyecto.



Vías de comunicación según la cantidad de personas en el proyecto: 1 vía con 2 personas, 3 vías con 3 personas, 6 vías con 4 personas, 10 vías con 5 personas y 45 vías con 10 personas.

Como puede verse en el gráfico, con un proyecto de dos personas hay una sola vía de comunicación. Al agregar una persona, la cantidad de vías aumenta linealmente a tres, pero al agregar una cuarta persona el crecimiento ya es mayor: cuatro personas generan seis vías de comunicación. Con cinco personas, las vías aumentan a diez, y con diez personas en el equipo, la cantidad de vías de comunicación llega a 45.

Con esto se observa que el crecimiento de la cantidad de vías de comunicación entre las personas que participan en un proyecto, a medida que crece la cantidad de personas, no es proporcional a ese crecimiento, sino que crece de manera exponencial.

La consecuencia de este aumento exponencial de las vías de comunicación (junto con otros factores) es que los proyectos también tienen un aumento exponencial del esfuerzo a medida que aumenta su tamaño. Esto se conoce como **deseconomía de escala**.

Fuera del software, solemos hablar de economías de escala y no de deseconomías de escala. Una economía de escala implica que cuanto más grande sea la producción, menor será el costo por unidad (por ejemplo: "si construimos una planta de fabricación más grande, podremos reducir el costo por unidad que producimos").

La deseconomía de escala es lo contrario: en el software, cuanto más grande es el sistema, mayor es el costo de cada unidad. Si el software presentara economía de escala, un sistema de 100.000 líneas de código (LOC) sería menos de 10 veces más costoso que uno de 10.000 LOC. Pero casi siempre ocurre lo contrario.

Video complementario: ver sección "Videos complementarios" al final del documento (video 5).

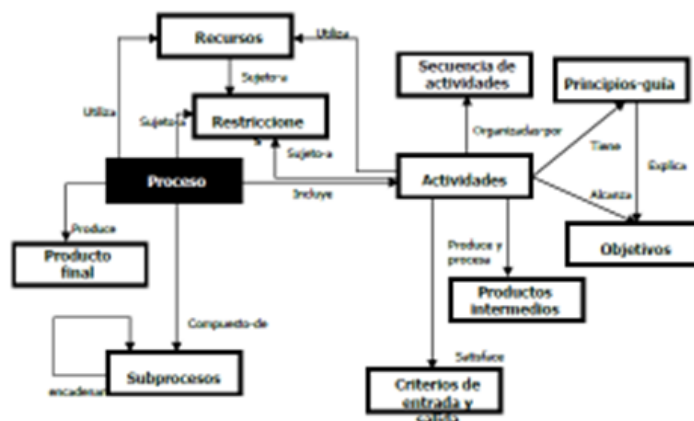
Concepto de proceso

- **IEEE:** "Una secuencia de pasos ejecutados para un propósito dado".
- Se ajusta a las prácticas específicas de las distintas ingenierías.
- Tiene dos componentes: acciones y propósito.
- La meta le da al proceso una serie de cualidades.
- **Pfleeger:** "Podemos pensar al conjunto ordenado de tareas como un proceso: una serie de pasos que involucran actividades, restricciones y recursos que producen una determinada salida esperada".
- **Basili:** afirma que los procesos de ingeniería del software son específicos, y así como el software como producto es único, no se aplican los mismos procesos para construir software que para construir cualquier otro producto.

El proceso en detalle (Pfleeger)

- Establece todas las principales actividades que involucra.
- Utiliza recursos, está sujeto a una serie de restricciones económicas y técnicas (por ejemplo, los recursos humanos pueden no estar capacitados para aplicar una cierta tecnología), y genera productos intermedios y finales.
- Puede estar compuesto por subprocesos que se encadenan de alguna manera.
- Cada actividad del proceso tiene criterios de entrada y salida: se ponen condiciones a los inputs para ser tratados por una determinada actividad (por ejemplo, un programa para ser aceptado en testing debe satisfacer los juegos de prueba provistos y el diseño definido).
- Las actividades se organizan en una secuencia.
- Tiene una serie de principios guía que explican los objetivos de cada actividad.
- Las restricciones o controles pueden aplicarse a una actividad, un recurso o un producto.

Mapa conceptual del proceso (Pfleeger)



Mapa conceptual del proceso según Pfleeger: el proceso utiliza recursos, está sujeto a restricciones, incluye actividades organizadas según una secuencia con principios-guía que alcanzan objetivos, produce productos intermedios que satisfacen criterios de entrada y salida, se compone de subprocesos y produce un producto final.

El proceso incluye actividades organizadas en una secuencia, con una serie de principios y guías. Esas actividades están diseñadas para alcanzar objetivos y pueden generar productos intermedios, siempre satisfaciendo una serie de criterios de entrada y salida.

Los procesos utilizan recursos y están sujetos a restricciones, tanto el proceso en su conjunto como las actividades que lo componen (que a su vez utilizan recursos y están sujetas a restricciones, por ser parte del proceso).

Si las actividades generan productos intermedios, el proceso como un todo produce o genera un producto final. Y, como ya se dijo, el proceso puede estar compuesto por subprocesos encadenados entre sí.

Esta visión de proceso ordena la generación de un producto. En el caso del software, existe un proceso que guía su desarrollo y que establece el ciclo de vida del software; para desarrollarlo hay distintos modelos de proceso.

Para el desarrollo de la ingeniería de requerimientos se tendrá un conjunto de actividades organizadas, donde se verá:

- La generación de productos intermedios.
- Criterios de entrada y salida de las actividades.
- Objetivos y utilización de recursos.
- Actividades sujetas a restricciones.
- Un proceso de ingeniería de requerimientos compuesto por subprocesos.
- Así como hay productos intermedios, un producto final por excelencia de la ingeniería de requerimientos.

En definitiva, así como se utiliza un proceso para el desarrollo de software, se utilizará un subproceso del desarrollo de software: el proceso de ingeniería de requerimientos. Este se verá como un proceso en sí mismo, compuesto a su vez por subprocesos, que generará productos intermedios y un gran producto final. Esta conceptualización del proceso no sólo es aplicable al software en general, sino también a un subproceso del desarrollo de software, como lo es la ingeniería de requerimientos.

Aquí finaliza esta clase. Se sugiere ver los videos complementarios indicados a continuación para completar los conceptos explicados.

Videos complementarios

No fue posible acceder al contenido de estos videos para resumirlos (no están indexados ni son reproducibles desde acá). Se listan en el orden en que aparecen en el material, junto con el tema que amplían según su ubicación en el texto original. Si compartís la transcripción de cada uno, puedo reemplazar esta referencia por un resumen real.

Video 1 — Ubicado tras la sección "El software en el tiempo": amplía el concepto de software como conjunto de información y no solo como programas.

<https://youtu.be/ZN1Q0fzf-jk?si=fi-0CSanFxRTiLIO>

Video 2 — Ubicado tras "Información en el ambiente de desarrollo": amplía las representaciones del software y el conocimiento de ingeniería de software.

<https://youtu.be/3aV0F2jMb2k?si=2wfUS-H289ccuxSM>

Video 3 — Ubicado tras "Características conceptuales": amplía la diferencia entre ingeniería clásica e Ingeniería de Software y la maleabilidad del software.

https://youtu.be/mwGgtwnLexQ?si=4F_rHie7M-rTf5Z4

Video 4 — Ubicado tras el listado de "Cualidades del software": amplía esas cualidades (corrección, confiabilidad, robustez, mantenibilidad, etc.).

https://youtu.be/kaXhVJzQnaY?si=1mjmVzbX_YbgH_Vq

Video 5 — Ubicado tras "El software y la diseconomía de escala": amplía el crecimiento exponencial de las vías de comunicación y su relación con el costo del proyecto.

https://youtu.be/fIL--Hu8t54?si=p2HZvWJPuKuba__D